1. Executable files

An executable file is a file containing instructions. These instructions must be in a format the CPU can understand. The standard format for the executable files on Linux is ELF[1] (*Executable and Linkable Format*). The format is defined in the `elf.h` header file. An executable file is a *binary* file, other binary files are *relocatable object files*, *core files*, *shared objects*.

1.1 The *Executable and Linkable Format* (ELF)

An ELF executable file consists of a an *ELF Header*, a *Program Header Table* or a *Section Header Table*, or both.

There are specific types defined for the ELF format which are listed in the following table[2]

`Elf32_Addr`	Unsigned program address, `uint32_t`
`Elf32_Off`	Unsigned file offset, `uint32_t`
`Elf32_Section`	Unsigned section index, `uint16_t`
`Elf32_Versym`	Unsigned version symbol information, `uint16_t`
`Elf_Byte`	`unsigned char`
`Elf32_Half`	`uint16_t`
`Elf32_Sword`	`int32_t`
`Elf32_Word`	`uint32_t`
`Elf32_Sxword`	`int64_t`
`Elf32_Xword`	`uint64_t`

Tab 1. ELF data types

For each type, both the 32-bit and 64-bit versions are defined. The same for the structures of the headers.

The *ELF Header* (*Ehdr*) is defined by the `Elf32_Ehdr` structure, which is defined as follows.

```
typedef struct
{
  unsigned char e_ident[EI_NIDENT]; /* Magic number and other info */
  Elf32_Half e_type;                /* Object file type */
  Elf32_Half e_machine;             /* Architecture */
  Elf32_Word e_version;             /* Object file version */
  Elf32_Addr e_entry;               /* Entry point virtual address */
  Elf32_Off e_phoff;           /* Program header table file offset */
  Elf32_Off e_shoff;           /* Section header table file offset */
  Elf32_Word e_flags;          /* Processor-specific flags */
  Elf32_Half e_ehsize;         /* ELF header size in bytes */
```

1 See the `elf(5)` manual page
2 In the `elf.h` header are also defined the 64-bit version of the data types

```
  Elf32_Half e_phentsize;      /* Program header table entry size */
  Elf32_Half e_phnum;          /* Program header table entry count */
  Elf32_Half e_shentsize;      /* Section header table entry size */
  Elf32_Half e_shnum;          /* Section header table entry count */
  Elf32_Half e_shstrndx;      /* Section header string table index */
} Elf32_Ehdr;
```

The 64-bit version of this structure is `Elf64_Ehdr`.

The *Program Header Table* (Phdr) is an array of structures found in *executable* and *shared object files*. These structures describe the segments and other information required by the system to load and prepare the program for execution. A *segment* in an *object file* may contain one or more sections.

The two fields `e_phentsize` and `e_phnum` of the `Elf32_Ehdr` structure specify the *program header* size.

The `Elf32_Phdr` structure is defined as follows.

```
typedef struct
{
   Elf32_Word p_type;          /* Segment type */
   Elf32_Off p_offset;         /* Segment file offset */
   Elf32_Addr p_vaddr;         /* Segment virtual address */
   Elf32_Addr p_paddr;         /* Segment physical address */
   Elf32_Word p_filesz;        /* Segment size in file */
   Elf32_Word p_memsz;         /* Segment size in memory */
   Elf32_Word p_flags;         /* Segment flags */
   Elf32_Word p_align;         /* Segment alignment */
} Elf32_Phdr;
```

The 64-bit version of the structure is `Elf64_Phdr`.

The *Section Header Table* (*Shdr*) is an array of `Elf32_Shdr` (or `Elf64_Shdr`) structures that allows to locate all the file's sections.
The `e_shoff` field of the ELF Header specifies the byte offset from the beginning of the file to the *Section Header Table* (Shdr).
The `e_shnum` field of the ELF Header (Ehdr) specifies the number of entries contained in the *Section Header Table*, while the `e_shentsize` field specifies the size, in bytes, of each entry.

Some array indices are reserved to represent special positions or predefined meanings.[3]
The `Elf32_Shdr` structure is defined as follows

```
typedef struct
{
   Elf32_Word sh_name;        /* Section name (string tbl index) */
   Elf32_Word sh_type;        /* Section type */
   Elf32_Word sh_flags;       /* Section flags */
```

3 These indexes are listed in the `elf(5)` manual page

```
   Elf32_Addr sh_addr;       /* Section virtual addr at execution */
   Elf32_Off sh_offset;      /* Section file offset */
   Elf32_Word sh_size;       /* Section size in bytes */
   Elf32_Word sh_link;       /* Link to another section */
   Elf32_Word sh_info;       /* Additional section information */
   Elf32_Word sh_addralign;  /* Section alignment */
   Elf32_Word sh_entsize;    /* Entry size if section holds table */
} Elf32_Shdr;
```

The 64-bit version of the structure is `Elf64_Shdr`.

In addition to the three main structures, ELF files also include the *Symbol Table* (Sym), the *Relocation Entries* (Rel and Rela), and the *Dynamic Tags* (Dyn).

The *object files Symbol Table* uses strings to represent *symbol* and *section names*. This table contains information to locate and relocate the symbolic definitions and references of a program, and is defined as follows.

```
typedef struct
{
   Elf32_Word st_name;       /* Symbol name (string tbl index) */
   Elf32_Addr st_value;      /* Symbol value */
   Elf32_Word st_size;       /* Symbol size */
   unsigned char st_info;    /* Symbol type and binding */
   unsigned char st_other;   /* Symbol visibility */
   Elf32_Section st_shndx;   /* Section index */
} Elf32_Sym;
```

The 64-bit version of the structure is `Elf64_Sym`.

Some symbols can have additional information that are stored in the `Elf32_Syminfo` structure, which is defined as follows.

```
typedef struct
{
   Elf32_Half si_boundto;    /* Direct bindings, symbol bound to */
   Elf32_Half si_flags;      /* Per symbol flags */
} Elf32_Syminfo;
```

The 64-bit version of the structure is `Elf64_Syminfo`.

The *Relocation Entries* are defined by the two structures `Elf32_Rel` and `Elf32_Rela`, where `Elf32_Rela` defines an entry that need an addend.

Relocation means to connect a *symbolic reference* to a *symbolic definition*. The *relocatable files* must have information that describes how to modify their section contents in order to allow the *executable* and *shared object files* to know the right information for a process's *program image*.
A *process's program image* is the program loaded into the memory. Once loaded it must know where to find the symbols it uses, this information is stored in the *relocation entries table*.

The `Elf32_Rel` structure is defined as follows.

```
typedef struct
{
   Elf32_Addr r_offset;      /* Address */
   Elf32_Word r_info;        /* Relocation type and symbol index */
} Elf32_Rel;
```

The 64-bit version of the structure is `Elf64_Rel`.

The `Elf32_Rela` structure is defined as follows.

```
typedef struct
{
   Elf32_Addr r_offset;      /* Address */
   Elf32_Word r_info;        /* Relocation type and symbol index */
   Elf32_Sword r_addend;     /* Addend */
} Elf32_Rela;
```

The 64-bit version of the structure is `Elf64_Rela`.

The *Dynamic Tags* (Dyn) contains structures holding information about the *dynamic linking*.
The `Elf32_Dyn` structure is defined as follows.

```
typedef struct
{
   Elf32_Sword d_tag;       /* Dynamic entry type */
   union
   {
      Elf32_Word d_val;     /* Integer value */
      Elf32_Addr d_ptr;     /* Address value */
   } d_un;
} Elf32_Dyn;
```

The 64-bit version of the structure is `Elf64_Dyn`.

Another structure used by ELF is the *Notes Header*. This contains arbitrary information used by the system. This data is largely used by *core files*. Also the *GNU tool chain* uses this header to pass information from the *linker* to the *C library*.
The `Elf32_Nhdr` structure is defined as follows.

```
typedef struct
{
   Elf32_Word n_namesz;      /* Length of the note's name. */
   Elf32_Word n_descsz;      /* Length of the note's descriptor. */
   Elf32_Word n_type;        /* Type of the note. */
} Elf32_Nhdr;
```

The 64-bit version of the structure is `Elf64_Nhdr`.

The `elf.h` header contains many other structures to specify all the features of the ELF format. The aforementioned are the most commonly used. The following list summarizes the structures we have seen so far.

Ehdr (ELF Header)
: Contains general information about the ELF file, such as file type, architecture, entry point address, and offsets to other structures

Phdr (Program Header Table)
: Describes the segments used by the system loader to prepare the program for execution

Shdr (Section Header Table)
: Describes the sections contained in the ELF file, such as .text, .data, and .bss

Sym (Symbol Table)
: Stores information about symbols, including functions and global variables

Syminfo (Symbol Information Table)
: Provides additional information about symbols, mainly used for dynamic linking and symbol resolution

Rel / Rela (Relocation Entries)
: Contain information required to modify addresses and references during linking or loading

Dyn (Dynamic Section/Tags)
: Contains information needed for dynamic linking, such as shared libraries and runtime linking data

Note (Notes Header/Section)
: Stores auxiliary information such as ABI details, build IDs, or vendor-specific metadata

1.2 The ELF Header

On Linux systems, there are many available tools to analyze *binary files:* `hexedit`, `objdump`, `gdb`, `readelf`, etc.. The `readelf` program reads and display the information contained in the ELF file passed as argument. We take a look at the *ELF Header* of the following program:

```
#include <stdio.h>
int main() {
      printf("Hello Linux World!!");
      return 0;
}
```

```
[linux@memory] readelf -h hello
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              DYN (Position-Independent Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x1050
  Start of program headers:          64 (bytes into file)
  Start of section headers:          13968 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         14
  Size of section headers:           64 (bytes)
  Number of section headers:         31
  Section header string table index: 30
```

The output displays the following fields:

Magic
: Identifies the file as ELF. Always starts with `0x7f 45 4c 46` (`\x7fELF`)

Class
: Whether the binary is 32-bit or 64-bit (ELF32 or ELF64)

Data
: Endianness, little-endian or big-endian

Version
: ELF format version (1 for current ELF)

OS/ABI
: Target ABI (e.g. UNIX System V, Linux, FreeBSD), usually "UNIX - System V" for Linux

ABI Version
: Version of the ABI[4]

Type
: The type of the ELF file
  - `ET_EXEC` → Executable
  - `ET_REL` → Relocatable object file (.o)
  - `ET_DYN` → Shared object (.so, PIE executable file)
  - `ET_CORE` → Core dump

Machine
: The target architecture (x86-64, ARM, RISC-V, etc.)

4 ABI stands for *Application Binary Interface* and defines the low-level interface between the binary code and the operating system, see the links in the *Bibliography* for further details

Version
: Version in hexadecimal format (`0x1`)

Entry point address
: The address where the execution begins (`0x1050`)

Start of program headers
: The file offset where the *program header table* begins, used by the kernel loader,

Start of section headers
: The file offset where the *section header table* begins, used by tools like `ld`

Flags
: Machine-specific flags, `0x0` for x86-64

Size of this header
: 64 for ELF64, 52 for ELF32

Size of program headers
: Size of one entry in the *program header table*

Number of program headers
: The number of the entries (*segments*)

Size of section headers
: The size of one entry in the *section header table*

Number of section headers
: The number of entries (*sections*)

Section header string table index
: Index of the section header that contains the section names

1.3 Sections and Segments

Passing the option `-S` we can retrieve the S*ection Headers*[5].

[linux@memory] readelf -S --wide hello

```
There are 31 section headers, starting at offset 0x3690:

Section Headers:
  [Nr] Name              Type            Address          Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL            0000000000000000 000000 000000 00      0   0  0
  [ 1] .note.gnu.property NOTE           0000000000000350 000350 000020 00   A  0   0  8
  [ 2] .note.gnu.build-id NOTE           0000000000000370 000370 000024 00   A  0   0  4
  [ 3] .interp           PROGBITS        0000000000000394 000394 00001c 00   A  0   0  1
  [ 4] .gnu.hash         GNU_HASH        00000000000003b0 0003b0 000024 00   A  5   0  8
  [ 5] .dynsym           DYNSYM          00000000000003d8 0003d8 0000a8 18   A  6   1  8
  [ 6] .dynstr           STRTAB          0000000000000480 000480 00008f 00   A  0   0  1
  [ 7] .gnu.version      VERSYM          0000000000000510 000510 00000e 02   A  5   0  2
  [ 8] .gnu.version_r    VERNEED         0000000000000520 000520 000030 00   A  6   1  8
  [ 9] .rela.dyn         RELA            0000000000000550 000550 0000c0 18   A  5   0  8
```

5 The `--wide` option ensures that the output fits on one line without wrapping to the next line

```
  [10] .rela.plt         RELA            0000000000000610 000610 000018 18  AI  5  24  8
  [11] .init             PROGBITS        0000000000001000 001000 000017 00  AX  0   0  4
  [12] .plt              PROGBITS        0000000000001020 001020 000020 10  AX  0   0 16
  [13] .plt.got          PROGBITS        0000000000001040 001040 000008 08  AX  0   0  8
  [14] .text             PROGBITS        0000000000001050 001050 000108 00  AX  0   0 16
  [15] .fini             PROGBITS        0000000000001158 001158 000009 00  AX  0   0  4
  [16] .rodata           PROGBITS        0000000000002000 002000 000018 00   A  0   0  4
  [17] .eh_frame_hdr     PROGBITS        0000000000002018 002018 00002c 00   A  0   0  4
  [18] .eh_frame         PROGBITS        0000000000002048 002048 0000ac 00   A  0   0  8
  [19] .note.ABI-tag     NOTE            00000000000020f4 0020f4 000020 00   A  0   0  4
  [20] .init_array       INIT_ARRAY      0000000000003dd0 002dd0 000008 08  WA  0   0  8
  [21] .fini_array       FINI_ARRAY      0000000000003dd8 002dd8 000008 08  WA  0   0  8
  [22] .dynamic          DYNAMIC         0000000000003de0 002de0 0001e0 10  WA  6   0  8
  [23] .got              PROGBITS        0000000000003fc0 002fc0 000028 08  WA  0   0  8
  [24] .got.plt          PROGBITS        0000000000003fe8 002fe8 000020 08  WA  0   0  8
  [25] .data             PROGBITS        0000000000004008 003008 000010 00  WA  0   0  8
  [26] .bss              NOBITS          0000000000004018 003018 000008 00  WA  0   0  1
  [27] .comment          PROGBITS        0000000000000000 003018 00001e 01  MS  0   0  1
  [28] .symtab           SYMTAB          0000000000000000 003038 000360 18     29  18  8
  [29] .strtab           STRTAB          0000000000000000 003398 0001dd 00      0   0  1
  [30] .shstrtab         STRTAB          0000000000000000 003575 00011a 00      0   0  1

Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
  L (link order), O (extra OS processing required), G (group), T (TLS),
  C (compressed), x (unknown), o (OS specific), E (exclude),
  D (mbind), l (large), p (processor specific)
```

The output is composed by the columns:

[Nr]
: The section index number (0 = reserved, NULL section)

Name
: The section name (`.text`, `.data`, `.bss`, `.symtab`, `.strtab`). the names come from the *section header string table*

Type
: The section type[6]

  `PROGBITS` → raw program data (code or constants)
  `NOBITS` → occupies memory but no space in file (`.bss`)
  `SYMTAB/DYNSYM` → symbol tables
  `STRTAB` → string tables
  `NOTE` → notes
  `REL/RELA` → relocation entries
  `NULL` → unused

Address
: The *virtual memory address* where the section is loaded at runtime

Off
: The file offset (in bytes) where this section begins inside the ELF file

Size
: The size of the section

6 The complete list is in the `elf.h` header file

ES (entry size)
: Size of each entry if the section holds a table of fixed-size entries, if it is not a table 0

Flg (flags)
: Section attributes[7]

  `A` → Alloc (section is loaded into memory)
  `X` → Executable (contains code)
  `W` → Writable
  `M` → Mergeable
  `S` → Strings (null-terminated)
  `I` → Info link used

Lk (link)
: Section index link

  For `SYMTAB` → points to the string table section index (`.strtab`)
  For `REL/RELA` → index of the section with symbols used for relocation

Inf (info)
: Extra information
  For relocation section → index of the section to which relocations apply
  For `SYMTAB` → number of the *section header table* index of the associated section

Al (align)
: Required alignment of section in memory (power of 2), e.g. `16` → address must be multiple of 16

To retrieve the *Program Headers* we can use the `-l` option.

```
[linux@memory] readelf -l --wide hello
Elf file type is DYN (Position-Independent Executable file)
Entry point 0x1050
There are 14 program headers, starting at offset 64

Program Headers:
  Type           Offset   VirtAddr           PhysAddr           FileSiz  MemSiz   Flg
Align
  PHDR           0x000040 0x0000000000000040 0x0000000000000040 0x000310 0x000310 R
0x8
  INTERP         0x000394 0x0000000000000394 0x0000000000000394 0x00001c 0x00001c R
0x1
      [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
  LOAD           0x000000 0x0000000000000000 0x0000000000000000 0x000628 0x000628 R
0x1000
  LOAD           0x001000 0x0000000000001000 0x0000000000001000 0x000161 0x000161 R E
0x1000
  LOAD           0x002000 0x0000000000002000 0x0000000000002000 0x000114 0x000114 R
0x1000
  LOAD           0x002dd0 0x0000000000003dd0 0x0000000000003dd0 0x000248 0x000250 RW
0x1000
  DYNAMIC        0x002de0 0x0000000000003de0 0x0000000000003de0 0x0001e0 0x0001e0 RW
0x8
  NOTE           0x000350 0x0000000000000350 0x0000000000000350 0x000020 0x000020 R
0x8
  NOTE           0x000370 0x0000000000000370 0x0000000000000370 0x000024 0x000024 R
0x4
```

7 The complete list is in the `elf.h` header file

```
  NOTE           0x0020f4 0x00000000000020f4 0x00000000000020f4 0x000020 0x000020 R
0x4
  GNU_PROPERTY   0x000350 0x0000000000000350 0x0000000000000350 0x000020 0x000020 R
0x8
  GNU_EH_FRAME   0x002018 0x0000000000002018 0x0000000000002018 0x00002c 0x00002c R
0x4
  GNU_STACK      0x000000 0x0000000000000000 0x0000000000000000 0x000000 0x000000 RW
0x10
  GNU_RELRO      0x002dd0 0x0000000000003dd0 0x0000000000003dd0 0x000230 0x000230 R
0x1

 Section to Segment mapping:
  Segment Sections...
   00
   01     .interp
   02     .note.gnu.property .note.gnu.buildid .interp .gnu.hash .dynsym .dynstr
          .gnu.version .gnu.version_r .rela.dyn .rela.plt
   03     .init .plt .plt.got .text .fini
   04     .rodata .eh_frame_hdr .eh_frame .note.ABI-tag
   05     .init_array .fini_array .dynamic .got .got.plt .data .bss
   06     .dynamic
   07     .note.gnu.property
   08     .note.gnu.build-id
   09     .note.ABI-tag
   10     .note.gnu.property
   11     .eh_frame_hdr
   12
   13     .init_array .fini_array .dynamic .got
```

The output is composed by the following columns:

Type
: The segment type that tells the loader what to do

  `LOAD` → Load this segment into memory
  `DYNAMIC` → Dynamic linking info
  `INTERP` → Path to dynamic linker (`ld.so`)
  `NOTE` → Auxiliary info (build-id, ABI tags)
  `PHDR` → Location of the *program header table* itself
  `GNU_STACK` → Stack permissions (`RW` vs `RWX`)
  `GNU_RELRO` → Read-only after relocation

Offset
: File offset of the segment in the ELF file

VirtAddr
: *Virtual memory address* where the segment will be mapped at runtime

PhysAddr
: Physical address (unused in Linux, matches VirtAddr)

FileSiz
: Number of bytes to read from the file

MemSiz

Number of bytes to allocate in memory, if `MemSiz > FileSiz` the extra space is zero-filled[8]

Flags

Memory permissions for the segment

`R` → Read
`W` → Write
`E` → Execute

Align

Alignment requirement

The second part of the output shows the mapping between the *Segments* and the *Sections*.

The *Section Headers* are used by the *linker* and *debuggers* while the *Program headers* are used by the *kernel loader* at runtime to map the binary into memory.

The *Sections* are: `.text`, `.data`, `.rodata`, `.bss`, `.symtab`, `.strtab`. Each *Section* has a specific type (e.g. code, data, symbol table) and associated flags that define its properties, such as read, write, or execute permissions. Sections are primarily used by the *linker* during the process of building the executable.

`.text`	Compiled code
`.data`	Initialized data
`.rodata`	Constants, strings
`.bss`	Uninitialized data
`.rel.text` `.rela.text`	Relocation info
`.symtab`	Symbol table
`.strtab`	String table

Tab 2. Sections of an executable

Sections are created in object files (`.o`) during the compilation process. Once the object files contain these sections, the linker (`ld`) merges sections of the same type from all object files to produce the final executable or shared library.
The *linker* also resolves the *symbols*, applies the relocations and discards the `.symtab` if not building with debug info (`-g` option of `gcc`).
After the linking process, *sections* are no longer needed by the kernel. Instead, they are mainly used by tools such as debuggers, disassemblers, and other analysis utilities.

Once the linking process is completed, an executable file is produced that can be loaded into memory by the *kernel loader.*

8 That is how the `.bss` section is created

The *kernel loader* reads the *Program Header Table* (segments) to map the segments. *Segments* are contiguous memory regions containing several *Sections* together and have permissions like read, write and execute.
PIE (*Position Independent Executable*) files are mapped into memory by treating the addresses specified in the *Program Header Table* as offsets from a randomized base address chosen by the kernel.

We compile the following code

```
#include <stdio.h>

#define MESSAGE "Hello Linux World!!"

int main() {
      printf(MESSAGE);
      return 0;
}
```

passing the `-S` option to `gcc`, in order to stop the compilation before assembling[9], and the option `-masm=intel` to use the Intel syntax for the code.

```
[linux@memory] gcc -masm=intel -S hello.c
```

The `hello.s` file is created.

```
      .file "hello.c"
      .intel_syntax noprefix
      .text
      .section .rodata
.LC0:
      .string "Hello Linux World!!"
      .text
      .globl main
      .type main, @function
main:
.LFB0:
      .cfi_startproc
      push rbp
      .cfi_def_cfa_offset 16
      .cfi_offset 6, -16
      mov rbp, rsp
      .cfi_def_cfa_register 6
      lea rax, .LC0[rip]
      mov rdi, rax
      mov eax, 0
      call printf@PLT
      mov eax, 0
```

9 In this way we can create and analyze the assembly file

```
        pop rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc
.LFE0:
        .size main, .-main
        .ident "GCC: (Debian 14.3.0-5) 14.3.0"
        .section .note.GNU-stack,"",@progbits
```

---

hello.s

We can see the *Sections* created by the compiler.

The `file` command executed on the `hello` executable give us information about the binary file.

```
[linux@memory] file hello
hello: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter
/lib64/ld-linux-x86-64.so.2, BuildID[sha1]=8bdc9b7b2228d196e34fe38dff9e9109ca5738d7, for
GNU/Linux 3.2.0, not stripped
```

The output shows that the program is an ELF 64-bit little-endian (LSB) PIE executable compiled for the x86-64 architecture. The binary is *dynamically linked*, meaning it depends on *shared libraries* at runtime, and it specifies the *dynamic linker* `/lib64/ld-linux-x86-64.so.2` used to load the program.
The output also includes a BuildID, which is a unique hash used to identify the binary version. It indicates compatibility with GNU/Linux 3.2.0 or later. Finally, the program is not stripped, meaning it still contains debugging symbols and metadata that can be used for analysis.

LSB	→	Least Significant Byte first (*little-endian*)
PIE	→	Position Independent Executable
not stripped	→	the ELF file still contains (after the compilation) the *section headers* (`.symtab`, `.strtab`, `.debug_*`) that are not required for the execution

To remove the *section headers* from the executable we can use the `strip`[10] command

```
[linux@memory] strip hello

[linux@memory] file hello
hello: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter
/lib64/ld-linux-x86-64.so.2, BuildID[sha1]=8bdc9b7b2228d196e34fe38dff9e9109ca5738d7, for
GNU/Linux 3.2.0, stripped
```

After running `strip`, the executable is marked as *stripped*, meaning that the section headers and most debugging-related information have been removed from the binary. As shown by the `file` output, the program remains fully functional as an ELF executable, but it no longer contains symbols and metadata useful for analysis.

10 See the `strip(1)` manual page

Section headers are mainly useful for development and analysis tools such as debuggers, disassemblers, and reverse engineering utilities. They are not required by the kernel at runtime, since the kernel relies on the *program headers* instead to load and map the executable into memory.

We can also see a difference between the size of the *not stripped* and *stripped* file

```
[linux@memory] ls -l hello
-rwxrwxr-x 1 io io 15952 Sep 30 18:18 hello

[linux@memory] strip hello

[linux@memory] ls -l hello
-rwxrwxr-x 1 io io 14464 Sep 30 18:19 hello
```